

Virtual Hand-Tracking Tic Tac Toe

Two-Player, Single-Camera Edition — Technical Documentation

Project	ICA Academy — Air Tic Tac Toe (MediaPipe Hand Tracking Demo)
Prepared for	Team Lead Review
Tech stack	Python, OpenCV, MediaPipe Tasks API (HandLandmarker), NumPy
Script type	Single-file desktop application (webcam-based)

1. Overview

This script implements a live, two-player game of Tic Tac Toe controlled entirely by hand gestures captured from a single webcam. It uses Google's MediaPipe HandLandmarker (the modern Tasks API) to track both players' hands simultaneously in real time, and renders a fully custom UI on top of the camera feed using OpenCV drawing primitives — there is no separate GUI framework involved.

Each player controls their mark by pointing an index finger at a cell and holding it in place for a short dwell time. The system distinguishes the two players automatically using MediaPipe's built-in handedness classification, so no manual pairing or setup step is required beyond both players being visible to the camera.

2. Requirements & Environment Setup

Python version compatibility — important: MediaPipe currently ships official wheels only for Python 3.9 through 3.12. There is no official MediaPipe wheel for Python 3.13 or 3.14 at this time. The script must therefore be run from a Python 3.12 virtual environment, even on machines whose system-default Python is newer (e.g. 3.13/3.14).

Setup (Windows example):

```
py -3.12 -m venv venv312
venv312\Scripts\activate
pip install mediapipe opencv-python numpy
```

Then run the script using that same virtual environment's **python.exe**.

Dependencies

- **opencv-python (cv2)** — video capture, drawing, and window management.
- **mediapipe** — hand landmark detection (Tasks API, VIDEO running mode).
- **numpy** — vector math for particles, colors, and smoothing.

- **tkinter** (standard library, optional) — used only to query the screen resolution so the window can auto-fit the display; falls back to a 1440×900 default if unavailable.

On first run, the script automatically downloads the required **hand_landmarker.task** model file from Google's public MediaPipe model storage if it is not already present in the working directory, so no manual model download step is needed.

Note: A working webcam at index 0 is required. If no camera is found, the script prints a clear error message and exits cleanly instead of crashing.

3. How Two-Player Tracking Works

The HandLandmarker is configured with **num_hands=2**, so both players' hands are tracked concurrently from the single webcam feed. For every detected hand, MediaPipe also returns a **handedness** label (Left / Right), which the script uses to make a fixed player assignment:

Detected Hand	Assigned Player	Mark	Accent Color
Left hand	Player 1	X	Orange
Right hand	Player 2	O	Yellow

Because the assignment is based on handedness rather than detection order, it stays stable even if a hand briefly leaves the frame and re-enters, or if hands appear in a different order between frames. The game remains strictly turn-based (standard Tic Tac Toe rules), but since both hands are tracked live, players do not need to physically hand over a mouse or keyboard — whichever player's turn it is simply points at an empty cell and holds their finger there for approximately **0.5 seconds** (*threshold_time*) to place their mark.

Fingertip coordinates (landmark index 8, the index-finger tip) are smoothed frame-to-frame with an exponential moving average (*CURSOR_SMOOTHING* = 0.35) to reduce jitter in the on-screen cursor.

4. Game State Machine

The application flow is governed by a simple state machine (*game_state*) with four states:

State	Description
waiting	Lobby screen with animated “Waiting for players...” dots. Held until both a left and a right hand are simultaneously detected.
countdown	A 3-2-1 countdown (<i>COUNTDOWN_SECONDS</i> = 3) once both hands are detected. Returns to “waiting” automatically when the countdown ends.
playing	Active round. The current player's hover position is tracked against the 3×3 grid; holding over an empty cell for 0.5 seconds places a mark.
won / draw	Round-end state. On a win, the winning line is highlighted and a particle celebration animation plays; the match then returns to the “waiting” state.

Win detection checks all 8 standard lines (3 rows, 3 columns, 2 diagonals). A full board with no winner transitions to the *draw* state. Pressing **r** resets the round while keeping match scores; pressing **m** starts a completely new match and resets scores to zero.

5. Key Features

- **Live “Player Detected” status badges** for both players, each with a pulsing green (detected) / red (not detected) indicator dot, so both people can instantly confirm the camera has locked onto their hand.
- **Lobby / waiting screen** with animated dots that only proceeds to the countdown once both hands are detected, preventing a round from starting with an undetected player.
- **Best-of-N match play** (default: first to 3 round wins takes the match), with a live match score panel and a “Match Point” callout badge when a player is one round away from winning.
- **Jitter-reduced cursors**: fingertip positions are smoothed with an exponential moving average so the on-screen pointer does not shake.
- **Particle-burst celebration animation** that plays over the winning line when a round is won.
- **Live FPS counter** and an optional full key-map help overlay (toggle with ‘h’).
- **Adaptive window sizing**: the window and board layout automatically scale to fit the user's screen resolution (capped at the original 1440×900 design size), fixing cases where part of the board was cut off on smaller laptop screens.
- **Friendlier startup / shutdown behavior**: a clear console error (instead of a silent crash) if no webcam is found, and an end-of-session summary (rounds played, final score, match winner) printed to the console on quit.

6. Controls

Key	Action
r	Restart the current round (match score is kept)
m	Start a brand-new match (resets match score to 0–0)
h	Show / hide the full keyboard shortcuts help panel
q	Quit the application

Interaction with the board itself is gesture-only: pointing at an empty cell and holding for the dwell threshold (0.5s) places the current player's mark. There is no mouse or keyboard interaction with the board itself.

7. Code Architecture

The script is organized as a single flat module, structured in the following logical sections (matching the numbered comment blocks in the source):

#	Section	Responsibility
1	Model bootstrap	Downloads hand_landmarker.task automatically if missing.
2	HandLandmarker setup	Creates the MediaPipe Tasks API landmarker in VIDEO mode with num_hands=2 and confidence threshold.
3	Theme & layout constants	ICA Academy color palette (BGR tuples), corner radius, dwell threshold, and auto-fit screen scaling logic.
4	Board layout	Computes the 3×3 grid geometry (cell size, gaps, position) and the 8 winning line combinations.
5	Game state	Global state: board contents, current player, game_state machine, scores, hover/dwell tracking, detection.
6	Drawing helpers	OpenCV-based UI rendering: rounded rectangles, gradient strip, pill badges, header, detection panel, etc.
7	Main loop	Per-frame pipeline: capture → flip/resize → darken/composite → run HandLandmarker → update state.

Notable functions

- **check_winner(b)** — scans all 8 win lines; returns (winning_player, winning_line) or (None, None).
- **reset_round(keep_scores)** — clears the board and hover/dwell state; returns to “playing” immediately if both hands are already detected, otherwise back to “waiting”.
- **start_new_match()** — resets round counter, match completion flag, and scores, then calls reset_round(keep_scores=False).
- **spawn_particles / update_particles / draw_particles** — lightweight physics-based particle burst (with gravity) used for the round-win celebration.
- **draw_detection_panel / draw_status_panel** — render the live per-player detection badges and the top status/lobby/countdown/turn/result panel respectively.

8. Configuration Reference

Key tunable constants defined near the top of the script:

Constant	Default	Purpose
<i>threshold_time</i>	0.5 s	Dwell time required to place a mark on a hovered cell.
<i>MATCH_TARGET</i>	3	Number of round wins needed to win the overall match.
<i>CURSOR_SMOOTHING</i>	0.35	EMA smoothing factor for the fingertip cursor (higher = snappier, less smoothing).
<i>COUNTDOWN_SECONDS</i>	3	Length of the pre-match “Get ready” countdown.
<i>CELEBRATION_SECONDS</i>	1.6	Duration/particle-lifetime range for the win-celebration burst.
<i>min_hand_detection_confidence</i>	0.4	MediaPipe HandLandmarker option — detection sensitivity.
<i>min_hand_presence_confidence</i>	0.5	MediaPipe HandLandmarker option — presence sensitivity.
<i>min_tracking_confidence</i>	0.5	MediaPipe HandLandmarker option — tracking sensitivity.

9. Known Limitations & Notes for Review

- **Single-camera dependency:** both players must share one webcam's field of view; there is no support for two separate camera feeds.
- **Handedness-based role assignment:** if a single player shows both hands (or players swap sides), the Left/Right → Player 1/2 mapping could momentarily misassign a mark. Physical seating (Player 1 on the camera's left, Player 2 on the right) is assumed.
- **Python version pinning:** the MediaPipe dependency currently caps supported Python at 3.12; this should be revisited if/when official 3.13+ wheels are released.
- **No persistence:** match/round history is only summarized to the console at exit and is not written to a file or database.
- **Model download on first run:** requires outbound network access to Google's model storage the first time the script runs on a machine (subsequent runs reuse the cached `hand_landmarker.task` file).

10. Suggested Next Steps

- Add configurable camera index / resolution selection for multi-camera setups.
- Persist match history (CSV/JSON) for longer play-testing sessions.
- Add automated tests around `check_winner()` and the state machine transitions.
- Evaluate MediaPipe Python 3.13 support once official wheels are published, to remove the 3.12-only constraint.

Document generated for internal review — ICA Academy Air Tic Tac Toe project.